→ Dot Product & cosine distance give nearly identical values. The reason for this is that the embedding is normalized (each vector has norm equal to 1). When the embedding is not normalized, the dot product & cosine distance would give different values.

* What is Attention in Language Models?

A huge roadback for language models is when a word can be used in two different contexts. When this prob^m is encountered, the model needs to use the context of the sentence in order to decipher which meaning of the word to use. This is precisely what self-attention models do.

Word embeddings have a huge Achilles heel: words that have more than one definition. If a word embedding assigns a vector to, say, the word 'bank', it assigns the same vector to all the definitions of 'bank'. What if you want to use this word in different contexts? Here is where attention comes into play.

Attention is a very clever way to tell words apart when they are used in different contexts (which turns word embeddings into contextualized word embeddings).

→ Dot Product & cosine distance give nearly identical values. The reason for this is that the embedding is normalized (each vector has norm equal to 1). When the embedding is not normalized, the dot product & cosine distance would give different values.

(*) What is Attention in Language Models?

A huge roadback for language models is when a word can be used in two different contexts. When this prob^m is encountered, the model needs to use the context of the sentence in order to decipher which meaning of the word to use. This is precisely what self-attention models do.

Word embeddings have a huge Achilles heel: words that have more than one definition. If a word embedding assigns a vector to, say, the word 'bank', it assigns the same vector to all the definitions of 'bank'. What if you want to use this word in different contexts? Here is where attention comes into play.

Attention is a very clever way to tell words apart when they are used in different contexts (which turns word embeddings into contextualized word embeddings)

(*) One word, multiple Meanings

In order to understand attention, let's look at 2 sentences

Sentence ① : The bank of the river.

Sentence ② : Money in the bank.

Attention

The bank of the river

Money in the bank

How would a computer know that the word "bank" in the 1st sentence refers to a setting in nature & in 2nd sentence to a financial setting?

Well let's ask a simpler question: How would a human know this? How did you figure out these 2 settings? The way you & I did it was probably to look at the neighbouring words. In the 1st sentence, the word "river" was the one that hinted at the nature setting & in the 2nd sentence, the word "money" was key to the financial setting. So, in short, we need a way to use the other words in the sentence to understand what context of the word "bank" we want to use.

Here is where embeddings are a each word. I lik that the words attached on a c contains all the 2 words that a close by. Now in are not exactly clo word "bank" and "river". Call this of the word " word "money". the following 2

- Modified s
- Modified

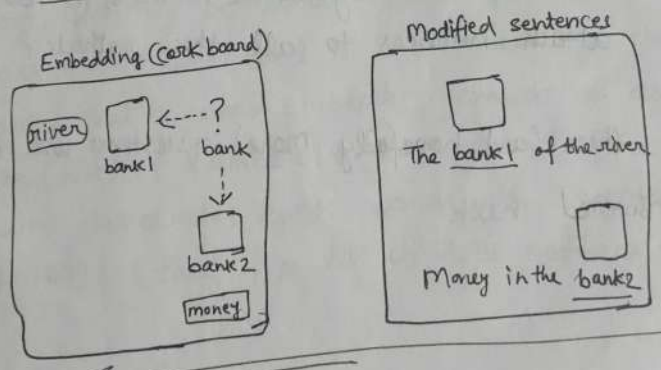
Embedding (

river

bank

Here is where word embeddings come into play. Word embeddings are a way to assign vectors (list of numbers) to each word. I like to imagine them geometrically. Imagine that the words "bank", "river" & "money" are all attached on a cork board. Furthermore, this cork board contains all the existing words & in such a way that 2 words that are similar (such as "apple" and "pear") are close by. Now in this cork board, "bank", "river" & "money" are not exactly close by. However, what you do is take the word "bank" and move it slightly towards the word "river". Call this word "bank1". Now, take another copy of the word "bank" and move it slightly towards the word "money". Call this one "bank2". Now, consider the following 2 modified sentences.

- Modified sentence 1 : The bank1 of the river.
- Modified sentence 2 : Money in the bank2.



In these 2 sentences, the computer now knows a little more about the context of the word "bank", as the word has been split into 2 distinct ones. One whose definition is closer to "river" and another one whose definition is closer to "money". That, in short, is how attention mechanism work. However there may be many questions lingering in your head. For example:

- What do you mean by "moving a word closer to another one"?
- Why did you ignore the other words in the sentence? How did you know the words "river" and "money" were the ones dictating the context, instead of the words "the", "In", "of"? As humans, we know which words provide context, but a computer would not have a clue.
- As computers only handle numbers, how do we attach numbers to all these methods?

All these (and hopefully more) questions will be answered next.

⇒ Moving words c

First, let me tell
word closer to and
this, is to average t
I want to move the
river. I now think c
That is, "bank1" i
Also, let's say t
"money". So, let'

- Bank1 = 0
- Bank 2 =

How did I come up
That comes later (ok
For now, you may
0.9 * Bank + 0.1 * Riv
we learned that a
a vector (list) of
embedding associ
length 4096 (th

⇒ Moving words on the Cork Board

First, let me tell you what I mean by "moving a word closer to another one". The way I like to imagine this, is to average two words. For example, let's say that I want to move the word "bank" 10% closer to the word river. I now think of the word "0.9_Bank + 0.1_River". That is, "bank1" is 90% "bank" & 10% "river". Also, let's say that "bank2" is 80% "bank" & 20% "money". So, let's say these are the modified words:

Bank1 = 0.9 * Bank + 0.1 * River

Bank 2 = 0.8 * Bank + 0.2 * Money

How did I come up with the numbers 0.9, 0.1, 0.8 & 0.2? That comes later (OK I have to spoil it, the answer is "similarity"). For now, you may be wondering what do I mean by 0.9 * Bank + 0.1 * River. Well, in the embeddings chapter, we learned that a word embedding consists of assigning a vector (list) of numbers to each word. The Cohere embedding associates each word with a vector of length 4096 (that is, a list of 4096 numbers per word).

For simplicity, let's imagine an embedding that associat
a vector of 2 numbers to each word :

- River : [0,5]
- Money : [8,0]
- Bank : [6,6]

These 2 numbers can be interpreted as coordinates in the plane, where the 1st no. is the horizontal coordinate, and the 2nd no., the vertical coordinate.

- Bank 1 : $0.9 * \text{Bank} + 0.1 * \text{River}$

$$= 0.9 * [6,6] + 0.1 * [0,5]$$
$$= [5.4, 5.4] + [0, 0.5]$$
$$= [5.4, 5.9]$$

- Bank 2 : $0.8 - \text{Bank} + 0.2 - \text{Money}$

$$= 0.8 * [6,6] + 0.2 * [8,0]$$
$$= [4.8, 4.8] + [1.6, 0]$$
$$= [6.4, 4.8]$$

↳ As you can see, "bank1" is closer to "river" & "bank2" is closer to "money". As a matter of fact, "bank1" is on the line b/w "bank" and "river", 10% along the way. Similarly, "bank2" is on the line b/w "bank" and "money", 20% along the way.

Thus the atte
the word "bank"
the corresponding

⇒ How to Decid

In other words, w
"money" instead of
determine the cont
"because I'm human
can the computer d
of them is a m
is multi-head att
Let's first discuss t
is going to do is to
as context, including
However, it's going
that amount is pr
"bank". We trust
b/w "bank" and
as they are uns
to ignore these
a higher simila

Thus the attention mechanism managed to split the word "bank" into 2 words & use each one in the corresponding sentence.

⇒ How to Decide which words determine context?

In other words, why did I pick the words "river" and "money" instead of "the", "of", and "in", in order to determine the context of "bank". Obviously the answer is "because I'm human and I know the language". But what can the computer do? It can rely on two mechanisms, one of them is a metric such as similarity, the second one is multi-head attention.

Let's first discuss the similarity mechanism. What the computer is going to do is to consider all the words in the sentence as context, including irrelevant words such as "the", "of", "in". However, it's going to consider them a certain amount, and that amount is precisely the similarity b/w the word and "bank". We trust that in a good embedding, the similarity b/w "bank" and a word such as "the" is almost zero, as they are unrelated. Therefore, the model will know to ignore these words & focus on those that may have a higher similarity with the word "bank".

Imagine that we calculate similarities for the words in each sentence & we get the following:

	the	bank	of	river
the	1	0	0	0
bank	0	1	0	0.11
of	0	0	1	0
river	0	0.11	0	1

The bank of the river

	money	in	the	bank
money	1	0	0	0.25
in	0	1	0	0
the	0	0	1	0
bank	0.25	0	0	1

Money in the bank

↳ Why is the similarity b/w "bank" and "money" higher than the similarity b/w "bank" & "river". We can imagine that "bank" gets used more often in the same context as "money", than as "river" and that explains the difference. We are simplifying this model quite a bit, it could be that the similarity b/w "the" and "of" is not zero, but 0.001.

Now, on to the next step. We are going to use the similarities to transform each of the words of this sentence. We'll call these new words "the1", "bank1", "of1", "river1" for the first sentence & "money2", "in2", "the2", "bank2" for the second sentence. Beware, we'll be doing some math with words, but it won't be very different from

the one we did before. Let's look at the first sentence, "The bank of the river", and the word "the". The similarities with the other words are as follows:

Similarities with "the":

- the : 1
- bank : 0
- of : 0
- river : 0

Therefore, we turn the word "the" into the new word "the1", which corresponds to the sum

1* "the" + 0*"bank" + 0* "of" + 0* "river".

This is equal to the word "the". That means, the word "the" does not chang & it's equal to "the1".

Now, let's follow the same procedure with the word "bank" & see what we get. The similarities of the word "bank" with the other words of the sentence "The bank of the river" are the following:

- the : 0
- bank : 1
- of : 0
- river : 0.11